



GitHub Copilot

Capítulo 2 – Skills, MCP y Multi-Agents

Marzo 2026

Repaso rápido – Capítulo 1

Lo esencial que debes recordar antes de continuar



LLM = Predicción

Copilot predice tokens, no piensa.
Tu contexto es su combustible.



Prompting Estructurado

ROL + CONTEXTO + TAREA +
RESTRICCIONES + FORMATO



Ventana de Contexto

El LLM solo 've' lo que cabe.
Archivos abiertos ~40% del espacio.



Instructions (.md)

Personal > Repo > Org. Configurar
una vez, consistencia siempre.



Agent Loop

Pensar → Actuar → Observar →
Repetir hasta completar la tarea.



Seguridad

Nunca secrets en el chat. Revisar
SIEMPRE código generado.

01



Prompt Files & Skills

Reutiliza conocimiento: escribe una vez, aplica siempre

¿Qué son los Prompt Files?

Plantillas reutilizables de prompts — control de versiones incluido



Archivos `.prompt.md` en tu repositorio

Viven en `.github/prompts/`. Se invocan con `/` en el chat. Contienen instrucciones predefinidas para tareas repetitivas.

```
.github/prompts/  
generate-tests.prompt.md  
review-code.prompt.md  
create-docs.prompt.md  
fix-cobol-bug.prompt.md
```

Generar Tests

Prompt file que genera tests unitarios adaptados al stack del proyecto (xUnit, NUnit, etc.)

Documentar Código

Estandariza la documentación: XML docs en C#, JSDoc en JS, comentarios en COBOL.

Review Checklist

Prompt file para revisión de código: seguridad, rendimiento, estándares del equipo.

Anatomía de un Prompt File

Ejemplo: generate-tests.prompt.md

```
---  
description: "Genera tests unitarios"  
mode: agent  
---
```

Actúa como un ingeniero de QA senior.
Stack: \${language} con \${testFramework}

Para cada función pública:

1. Genera caso happy path
2. Genera caso edge/error
3. Nombra tests con convención:
 Metodo_Escenario_ResultadoEsperado

Ejecuta los tests y corrige errores.

Front Matter (YAML)

description, mode (ask/agent/edit),
variables con \${}

Cuerpo (Markdown)

El prompt en sí: rol, instrucciones paso a
paso, formato esperado.

Invocación

/generate-tests en el chat de Copilot



Los Prompt Files se comparten entre IDE, CLI y GitHub.com. ¡Un solo archivo, tres superficies!

Skills personalizadas

Prompt Files + Context = Skills reutilizables para tu equipo

Prompt File

Instrucciones estáticas en Markdown

Variables con \${} para personalizar

Se invocan con / en el chat

Ideal para: "Genera tests", "Documenta"

Skill (Prompt File + Contexto)

Prompt + archivos de referencia adjuntos

Puede incluir ejemplos, schemas, reglas

Encadena herramientas (MCP, tests)

Ideal para: workflows complejos del equipo

Ejemplo de Skills para GCO:

Skill COBOL: genera copybook desde spec • Skill .NET: crea endpoint REST con validación • Skill Tests: genera tests + ejecuta + reporta

02



MCP – Model Context Protocol

Conecta Copilot a fuentes externas: bases de datos, APIs, docs

¿Qué es MCP?

Model Context Protocol — El 'USB-C' de los agentes de IA



Un protocolo estándar y abierto para conectar LLMs con fuentes de datos externas

MCP define cómo un agente (Copilot, Claude Code, Cursor...) descubre y utiliza herramientas externas. El LLM decide cuándo llamar a qué herramienta, sin que tú tengas que programar la integración.

Analogía: Si el LLM es el cerebro y las tools son extremidades, MCP es el sistema nervioso que las conecta.

Copilot (Cliente)



MCP Server



Tu Fuente de Datos

DB · API · Docs · Git · Elastic...



MCP es agnóstico del LLM: funciona igual con Copilot CLI, VS Code, Claude Code o Cursor Agent.

Configuración MCP en Copilot

Dos formas de conectar MCP servers a Copilot

VS Code / IDE

```
// .vscode/mcp.json
{
  "servers": {
    "my-db-server": {
      "type": "stdio",
      "command": "npx",
      "args": ["mcp-server-db"]
    }
  }
}
```

Copilot CLI

```
// .github/copilot/mcp.json
{
  "mcpServers": {
    "github": {
      "type": "stdio",
      "command": "npx",
      "args": [
        "@anthropic/mcp-gh"
      ]
    }
  }
}
```

Ambos archivos se versionan con el repo. Todo el equipo comparte la misma configuración MCP.

Casos de uso MCP

Conecta Copilot a las herramientas que usas cada día



Base de Datos

Conecta a SQL Server / Oracle. Copilot lee el schema y genera queries, clientes, migraciones.



Elastic / Logs

Busca errores en logs directamente desde el chat. Copia el JSON del error y Copilot investiga.



GitHub MCP

Acceso a issues, PRs, workflows, code search. Ya integrado en Copilot CLI.



Documentación

Indexa tu wiki/Confluence. Copilot responde preguntas basándose en docs de tu equipo.



APIs Internas

Expón tus endpoints como MCP tools. Copilot puede llamar APIs para consultar o validar.



Custom MCP Server

Crea tu propio server con Node o Python. Define tools específicas de tu proyecto.

Ejercicio 1 - Skill COBOL: Generar Tests Estándar

1. Crea un archivo `.github/prompts/cobol-test.prompt.md` en tu repositorio
2. Define el prompt: rol de tester COBOL senior, genera casos de test para el programa, incluyendo datos de entrada y salida esperada
3. Invoca `/cobol-test` en el chat con un programa COBOL abierto
4. Revisa – ¡LA RESPONSABILIDAD ES SÓLO TUYA !
5. Itera el prompt file: ajusta instrucciones, añade formato de salida
6. Comparte el prompt file con tu equipo vía Git

Ejercicio - Skill .NET: Conectar MCP a Base de Datos

1. Configura un MCP server de base de datos en `.vscode/mcp.json`
(ej: `npx @anthropic/mcp-server-sqlite` o similar para SQL Server)
2. Pide a Copilot que explore el schema de la base de datos
3. Genera un cliente/repositorio .NET basado en el schema real
4. Revisa – ¡LA RESPONSABILIDAD ES SÓLO TUYA !
5. Pide a Copilot que genere queries y valide contra el schema
6. Itera: ajusta el schema, regenera, compara resultados

03



Arquitectura Multi-Agents

Orquesta múltiples agentes especializados para tareas complejas

Roles en Multi-Agents

Cada agente tiene un rol específico — como un equipo de desarrollo real



Arquitecto

Analiza requisitos, diseña la solución, define la estructura de archivos y dependencias.



Implementador

Escribe el código siguiendo el plan del arquitecto. Puede trabajar en paralelo con otros.



Tester

Genera y ejecuta tests. Valida que la implementación cumple con la especificación.



Revisor

Code review automatizado: seguridad, rendimiento, estándares del equipo, SOLID.

En Copilot CLI: usa sub-agentes con `/task`. En VS Code: usa Custom Agents (`.prompt.md` con roles definidos).

Flujo Multi-Agent

Orquestación paso a paso con validación cruzada



← **ITERAR** si el Revisor o Tester encuentra problemas →

¿Cómo se implementa en la práctica?

Copilot CLI: usa `/task` para lanzar sub-agentes que trabajan en paralelo. Cada sub-agente recibe un prompt específico con su rol y contexto. El agente principal coordina y valida los resultados antes de aplicar cambios.

Ejemplo práctico: Feature completa

"Añadir endpoint REST para consultar histórico de facturas"

Tú Defines la spec en lenguaje natural o en un .prompt.md con las reglas de negocio

Arquitecto Analiza el codebase, propone: controlador, servicio, repositorio, DTOs, y plan de archivos

Implementador Genera el código en C# / .NET 8 siguiendo el plan. Crea archivos, ajusta DI container

Tester Genera tests xUnit/NUnit. Ejecuta dotnet test. Corrige errores hasta pasar

Revisor Valida: naming, SOLID, seguridad, rendimiento. Propone cambios y el ciclo se repite

"Multi-agent no es magia: es dividir una tarea grande en tareas pequeñas con roles claros."

Ejercicio 2 - COBOL Multi-Agent: Feature Evolutiva

1. Selecciona un evolutivo COBOL pendiente (nueva funcionalidad o mejora)
2. Escribe una spec clara en modo Ask: qué hace, qué datos usa, qué salidas
3. Pide al agente que actúe como "Arquitecto": que analice el código existente y proponga un plan de cambios antes de tocar nada
4. Revisa el plan – ¡LA RESPONSABILIDAD ES SÓLO TUYA !
5. Pasa a modo Agente para que implemente los cambios
6. Pide que genere casos de test y valide contra la spec

Ejercicio 2 - .NET Multi-Agent: Implementar + Test + Review

1. Selecciona una tarea .NET pendiente (endpoint, servicio, bug fix)
2. Usa Plan mode (Shift+Tab) para que Copilot analice y planifique
3. Ejecuta la implementación en modo Agente
4. Pide al agente que genere tests unitarios y los ejecute
5. Pide una revisión de código: seguridad, rendimiento, SOLID
6. Itera: corrige lo que el reviewer sugiere, vuelve a pasar tests
7. Revisa – ¡LA RESPONSABILIDAD ES SÓLO TUYA !

04



Integración con Datos

Bases de datos, documentación y fuentes de verdad

Conectando Copilot a tus datos

MCP como puente entre tu código y tus fuentes de verdad



Base de Datos vía MCP

Copilot lee el schema real de tu BD

Genera queries SQL correctos (no inventados)

Crea repositorios/clientes que coinciden con tablas reales

Reduce alucinaciones: el schema es la fuente de verdad



Documentación vía MCP

Indexa tu wiki, Confluence, SharePoint

Copilot responde basándose en docs del equipo

Mantén la doc actualizada = mejores respuestas

Onboarding acelerado: la IA conoce tu proyecto



Recuerda: MCP no envía tu BD completa al modelo. El MCP Server expone herramientas (tools) que el agente invoca bajo demanda. Solo se envía la información que el agente solicita en cada paso del agent loop.

En resumen...



Prompt Files

Escribe tus prompts una vez, versiónalos, compártelos. /generate-tests > reescribir cada vez.



Skills

Prompt + contexto + herramientas = workflows reutilizables para todo el equipo.



MCP

Conecta Copilot a BD, APIs, docs. El agente consulta fuentes de verdad, no inventa.



Multi-Agents

Divide tareas complejas en roles: Arquitecto, Implementador, Tester, Revisor.



Valida siempre

Más poder = más responsabilidad. Revisa cada paso, especialmente con acceso a datos.



Experimenta

Prueba combinaciones: MCP + Prompt File + Multi-Agent. La IA evoluciona cada semana.

Próximo capítulo: Spec-Driven Development

Capítulo 3 — Nivel Experto



Especificaciones formales

Diseña specs que guían al agente. Menos ambigüedad = menos alucinaciones.



Tests antes de código

Genera tests desde la spec. Implementa hasta que pasen. TDD potenciado.



Validación automática

El agente verifica que el código cumple la spec. Cobertura y reglas arquitectónicas.



Multi-Agent + Spec

Combina todo: spec → plan → implementar → test → review, orquestado.

<https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai/>

*" De chatbot a orquestador: la IA ya no solo responde,
ahora actúa, conecta y valida. "*

¿Preguntas?

¡Gracias!